Internship report - 13.06.-30.09.

Javier

September 15, 2026

1 Introduction

This study investigates the optimization of simultaneous multi-robot path planning: finding collision-free paths for a set of agents on a shared graph or grid that minimize the total time required for all agents to reach their respective goals. The problem is formulated as a single joint combinatorial optimization problem over all agents, in contrast to decomposition-based approaches such as Conflict-Based Search (CBS) or MA-RRT, which plan agents individually and resolve conflicts through iterative search.

We study two joint formulations of this problem: Quadratic Unconstrained Binary Optimization (QUBO), motivated by potential adiabatic-based solvers (annealers and QAOA), and an equivalent Integer Linear Programming (ILP) formulation, solvable by classical exact solvers. We use both to benchmark against decomposition-based methods. Unlike naive joint-state search, whose state space grows exponentially with the number of agents, both QUBO and ILP represent the joint problem compactly: their variable count scales only linearly with the number of agents.

Note that this does not eliminate the underlying NP-hardness of multi-agent pathfinding; worst-case solve time can still grow sharply with agent count under both annealing and branch-and-cut, but it avoids the explicit combinatorial blowup in problem representation that naive joint approaches suffer from. Decomposition methods such as CBS avoid this state-space blowup through conflict-tree search, but can themselves incur exponential branching in the number of conflicts, motivating a direct comparison against compact joint formulations.

We assume a discretized environment (a grid or graph) and a discretized time horizon, with each agent's start and goal position fixed and known in advance; agents move at most one edge per time step and must avoid colliding with one another.

Autonomous multi-agent navigation is a rapidly advancing and compelling research area in the robotics field. Multi-robot systems are increasingly deployed

in warehouse automation, inspection, and distributed sensing tasks, wherein multiple agents must navigate shared environments efficiently and safely. These systems offer significant advantages and a broader range of possibilities, but introduce greater planning and coordination complexity.

We pursue optimal collision-free paths for multiple agents in shared environments while also addressing quantum computation techniques in order to get better solvers and more efficient solutions.

2 ILP/BLP/MILP formulation

2.1 Modeling idea

This work considers a multi-agent (single-agent is an special case) path planning problem for robotics applications.

A set of agents is defined. Each agent has a fixed start position, end position, and a local time-window where it is free to move. They can move to whenever cell is considered the best; however there are some restrictions

- Only valid movements are allowed,
- Valid movements are defined by connectivity and Neighbour set (obstacles are excluded from that list)
- Additionally, in multi-robot, a movement is invalid if two or more robots move through the same edge or stay in the same vertex
- Agents can only be at one cell per-timestep, hence you can only advance one vertex at a time
- Agents can only move during their active window time

It is important to note that during the agent's entire active window, that is, from its initial time to its latest arrival deadline, including any time spent waiting at the goal after arrival, the agent occupies a physical space in the map and can crash with other agents. Only once that window ends, the agent deactivates and stops occupying a position in the map.

Agent could or could not stay in the same cell depending on the Neighbor set, that is, if you include your own cell as a neighbor, transition to yourself counts as staying; however, in QUBO formulation so far I have been working with staying on cell restriction, this way you explicitly tell the solver to move and advance toward the goal (so to get a better path). In ILP this probably would be less effective and have more downsides so staying in place should be added.

As a recommendation, in single-robot scenarios, staying in place will almost never produce the optimal path so its simpler to remove it from neighbor set. In multi-robot, depending on environment, it may be required to allow staying in place.

2.2 Sets

We formulate the problem with three fundamental sets: the agent set A (normally representing robots), the environment set G (a grid or graph in which agents move), and a discrete time horizon $\mathcal{T}$ over which agent trajectories are planned.

- **Set of Agents**

$$A = \{a_1, \ldots, a_n\}, \qquad n \in \mathbb{N}$$

- **Environment Set (grid or graph)**

$$G = (V, E), \quad V = \{v_1, \ldots, v_k\}, k \in \mathbb{N}, \quad E \subseteq V \times V$$

  Grids are treated as a special case of graphs, where each cell corresponds to a vertex $v \in V$, and edges connect cells that are reachable from one another in a single agent move (e.g., 4- or 8-connectivity). We treat E as undirected, that is, $(v_i, v_j) \in E \iff (v_j, v_i) \in E$.

- **Discrete Time Horizon**

$$\mathcal{T} = \{0, 1, \ldots, T-1\}, \qquad T \in \mathbb{N}$$

$$\mathcal{T}^- = \{0, \ldots, T-2\}, \qquad T \in \mathbb{N}$$

Additionally, for the simplification of rules and penalties, we define a *neighborhood function* that specifies all valid single-step transitions between vertices. This function is used both to enforce movement constraints and as the basis for pre-processing techniques such as reachability pruning.

$$N(v) = \{v' \in V : (v, v') \in E\}, \qquad \forall v \in V$$

Lastly, for implementation we use two more sets

- **Set of Constants**

$$\mathcal{K} = \{K_{hot}, K_{adj}, K_{start}, K_{goal}, K_{crash}, K_{swap}, \ldots\}, \qquad K_i \in \mathbb{R}_{>0}$$

- **Set of Active Agents**

$$\mathcal{A}_t \subseteq A, \qquad \mathcal{A}_t = \{a \in A : t \in [t_a^{start}, t_a^{end}]\}, \qquad \forall t \in \mathcal{T}$$

2.3 Parameters

Agent parameters Every agent $a \in A$ has

- a start vertex, given as a fixed input for the problem instance

$$s_a \in V$$

- a goal vertex, given as a fixed input for the problem instance [1]

$$g_a \in V$$

[1] The case $s_a = g_a$ is permitted; the agent's sub-problem is then trivially satisfied at $t = t_a^{start}$.

Every agent's start and goal are treated as parameters of a given problem instance rather than decision variables (any valid vertex $v \in V$ may be assigned as such). That is, they are values known before execution and stay fixed. Note that these are distinct from the position variable v_a^t introduced in subsection 2.4, which denotes the vertex occupied by agent a at time t and is determined by the optimization.

Temporal parameters In the multi-agent setting, every agent $a \in A$ additionally has

- an earliest departure time, before which the agent may not leave s_a

$$t_a^{\text{start}} \in \mathcal{T}$$

- a latest arrival time, by which the agent must reach g_a

$$t_a^{\text{goal}} \in \mathcal{T}$$

For all agents $a \in A$,

$$t_a^{\text{goal}} \geq t_a^{\text{start}}$$

is required.

2.4 Decision Variables

Position variables For every agent $a \in A$, vertex $v \in V$, and time step $t \in \mathcal{T}$, let

$$x_{a,v,t} = \begin{cases} 1 & \text{, if agent } a \text{ occupies vertex } v \text{ at time step } t, \\ 0 & \text{, otherwise} \end{cases}$$

For notational convenience, we write v_a^t to denote the (unique) vertex satisfying $x_{a,v_a^t,t} = 1$, i.e., the position of agent a at time t as determined by the optimization.
Note that this variable set has size $|A| \times |V| \times |\mathcal{T}|$.

2.5 Objective

The problem objective is to minimize the total time each agent requires to reach its goal:

$$\min \sum_{a \in A} \tau_a, \qquad \tau_a = \min\{t \in \mathcal{T} : x_{a,g_a,t} = 1\} \tag{1}$$

Under the modelling assumption that each agent remains at its goal once reached, and assigning every cell equal weight, this is equivalent to minimizing the total time spent *away* from the goal. In the QUBO formulation this behavior is only encouraged, via a soft reward; in the ILP formulation it is enforced exactly as a constraint (Section 2.6.5), which guarantees the equivalence below holds exactly rather than approximately.

Note that the agent remains *active* at its goal for the remainder of its time window: it continues to occupy space and is subject to the same crash- and swap-avoidance constraints as any other agent, rather than being removed from the map.

$$\min \sum_{a \in A} \sum_{t \in \mathcal{T}} \sum_{v \in V \setminus \{g_a\}} x_{a,v,t} \tag{2}$$

where we consider every cell to have the same weight in the environment.

In summary, the objective is to find the globally optimal joint plan across all agents, that is, the shortest feasible set of routes satisfying the constraints introduced in Section 2.6 (adjacency, collision, swap-avoidance, among others).

2.6 Constraints

In QUBO formulation we divided constraints into two types: core constraints and helper constraints. However, when reformulating for ILP we can drop all the helper constraints and keep only the core, since here they aren't objective functions but rules to satisfy, you do not need extra-tweaks.
Core constraints are those that are required to be satisfied in order to get a valid answer. In single-agent environment there are 4 penalties of this type

2.6.1 One-Hot

No agent can be active in more than one position per time-step.
(Note one-hot is not necessary for all encodings)

QUBO Formulation

$$P_{hot} = K_{hot} \cdot \sum_{a \in A} \sum_{t \in \mathcal{T}} (1 - \sum_{v \in V} x_{a,v,t})^2 \tag{3}$$

The linear equivalent (as a hard constraint, not objective) would be

$$\sum_{v \in V} x_{a,v,t} = 1 \quad \forall a \in A, t \in \mathcal{T} \tag{4}$$

Note that in this case a is fixed per agent, evidently there can be more than one agent active

2.6.2 Adjacency

Agents can only move to valid neighbor cells.
It relies on the neighborhood previously defined

$$P_{adj} = K_{adj} \cdot \sum_{a \in A} \sum_{t \in \mathcal{T}^-} \sum_{v \in V} x_{a,v,t}(1 - \sum_{(v') \in N(v)} x_{a,v',t+1}) \tag{5}$$

Note that it could also be equivalent to penalizing all non-neighbor vertex; however, that alternative requires much more iterations

$$P_{adj} = K_{adj} \cdot \sum_{a \in A} \sum_{t \in \mathcal{T}^-} \sum_{v \in V} \sum_{v' \notin N(v)} x_{a,v,t}\, x_{a,v',t+1}, \tag{6}$$

The linear constraint equivalent (it is the same for both schemes) would be

$$x_{a,v,t} \leq \sum_{v' \in N(v)} x_{a,v',t+1} \quad \forall v \in V, \forall a \in A, \forall t \in \mathcal{T}^- \tag{7}$$

2.6.3 Start

Enforces the agent to start on the start cell.

$$P_{start} = -K_{start} \cdot \sum_{a \in A} x_{a,s_a,t_a^{start}} \tag{8}$$

It is technically already linear, so we just rewrite it in ILP format

$$x_{a,s_a,t_a^{start}} = 1 \qquad \forall a \in A \tag{9}$$

2.6.4 Goal/End

Forces the agent to reach the goal before the window finishes.

$$P_{end} = -K_{end} \cdot \sum_{a \in A} x_{a,g_a,t_a^{goal}} \tag{10}$$

The same as before, we simply need to rewrite in ILP format

$$x_{a,g_a,t_a^{goal}} = 1 \qquad \forall a \in A \tag{11}$$

2.6.5 Goal Lock

Once the agent reaches the goal it should stay there until the windows finishes

In the case of QUBO, goal lock is what I call a soft-penalty (not necessary but useful, since it can be fixed on post-processing). However, in ILP, it is easier to enforce goal lock, and it would be define as a constraint

The QUBO format is

$$P_{lock} = K_{lock} \cdot \sum_{a \in A} \sum_{t \in \mathcal{T}^-} x_{a,g_a,t}(1 - x_{a,g_a,t+1}) \tag{12}$$

The linear equivalent would be

$$x_{a,g_a,t} \leq x_{a,g_a,t+1} \qquad \forall a \in A, \forall t \in \mathcal{T}^- \tag{13}$$

For multi-agent environment we keep those 5 penalties per robot (locally), but we need to add 2 more constraint

2.6.6 Crash

No two agents can be on the same position at the same timestep.

$$P_{crash} = K_{crash} \cdot \sum_{v \in V} \sum_{\substack{a_i, a_j \in \mathcal{A}_t \\ i<j}} \sum_{t \in \mathcal{T}} x_{a_i,v,t} \cdot x_{a_j,v,t} \tag{14}$$

The ILP equivalent would be

$$\sum_{a \in \mathcal{A}_t} x_{a,v,t} \leq 1 \qquad \forall v \in V, \forall t \in \mathcal{T} \tag{15}$$

2.6.7 Swap

No two agents may exchange positions across an edge in a single time step.

Exact formulation. The true swap condition is a degree-4 term:

$$P_{swap} = K_{swap} \sum_{(v,w) \in E} \sum_{\substack{a_i, a_j \in \mathcal{A}_t \\ i<j}} \sum_{t \in \mathcal{T}^-} x_{a_i,v,t}\, x_{a_j,w,t}\, x_{a_i,w,t+1}\, x_{a_j,v,t+1} \tag{16}$$

Encoding this directly in QUBO form requires quadratizing a degree-4 term, which introduces multiple chained auxiliary variables, which becomes particularly costly for the solvers.

Quadratic approximation (current implementation). To avoid this cost, we instead use a quadratic proxy:

$$P_{swap_approx} = K_{swap} \cdot \sum_{v \in V} \sum_{\substack{a_i, a_j \in \mathcal{A}_t \\ i<j}} \sum_{t \in \mathcal{T}^-} (x_{a_i,v,t}\, x_{a_j,v,t+1} + x_{a_i,v,t+1}\, x_{a_j,v,t} + x_{a_i,v,t}\, x_{a_j,v,t}) \tag{17}$$

This remains purely quadratic but does not exactly encode the swap condition; it is best understood as a soft, imperfect deterrent that partially overlaps with crash-avoidance, rather than an exact constraint.
(Note that last term is actually the same as crash collision, in practice we implement crash + swap, but applying K_{swap} to first two terms and K_{crash} to third term).

ILP formulation. Since ILP constraints are not restricted to degree ≤ 2 and do not require quadratization, the exact swap condition can be enforced directly and cheaply as a linear constraint, without auxiliary variables:

$$\begin{aligned} &x_{a_i,v,t} + x_{a_j,w,t} + x_{a_i,w,t+1} + x_{a_j,v,t+1} \leq 3 \\ &\forall (v,w) \in E, \quad \forall\, a_i, a_j \in \mathcal{A}_t \text{ with } i < j, \quad \forall t \in \mathcal{T}^- \end{aligned} \tag{18}$$

to

$$\begin{aligned} &x_{a_i,v,t} + x_{a_j,w,t} + x_{a_i,w,t+1} + x_{a_j,v,t+1} \leq 3 \\ &\forall (v,w) \in E, \quad \forall\, a_i, a_j \in \mathcal{A}_t \text{ with } i < j, \quad \forall t \in \mathcal{T}^- \end{aligned} \tag{19}$$

3 QUBO formulation

4 Optimization Techniques

For a classical solver, minimizing the objective directly with uniform cell weights is sufficient, the search procedure itself (e.g., BFS, A*) finds the shortest feasible path without requiring any additional guidance. For QUBO-based solvers (QAOA, quantum annealing), however, we find empirically that a flat, uniform objective yields a poorly-shaped energy landscape, providing weak guidance towards the goal, particularly on larger environments.

We therefore introduce a potential-based reward shaping scheme:

$$\sum_{a \in A} \sum_{t \in \mathcal{T}} \sum_{v \in V \setminus \{g_a\}} c_v \, x_{a,v,t} \tag{20}$$

where $c_v \in \mathbb{R}_{>0}$ denotes the cost of occupying vertex v, combining distance-to-goal, time-dependent amplification, and obstacle repulsion terms. Goal cells receive a negative weight (a *reward*), while non-goal cells receive weights (*penalties*) that depend on connectivity, distance to the goal, and proximity to obstacles.

This shaping is designed to preserve the ranking of feasible solutions under the formal objective in Section **??**, while substantially improving solver convergence in practice. The approach is conceptually similar to potential-based reward shaping in reinforcement learning, and we believe further connections to that literature may help formalize and improve the QUBO construction.

4.1 From QUBO Heuristics to ILP: What Transfers and Why

Several penalty terms present in our QUBO formulation do not appear as constraints in the ILP formulation above. This is not an oversight, but a direct consequence of ILP enforcing constraints *exactly*, whereas QUBO-based solvers only encourage constraint satisfaction through the energy landscape (objective functions), without guaranteeing it prior to convergence.

Term	Core in ILP?	Reasoning
Backtracking	No	Already discouraged by the objective in the single-agent case; enforcing it as a hard constraint risks infeasibility in multi-agent settings.
Obstacles	No (handled by preprocessing)	Removing obstacle cells from V / excluding them from $N(v)$ is strictly better: avoidance is guaranteed structurally, with no penalty term required.
Teleportation (TP)	No	Prevents reaching a goal faster than its Manhattan distance allows; redundant once adjacency is enforced as an exact hard constraint, since position can then change by at most one edge per time step.
Goal-approximation	Optional	Not required for correctness; retained only as an optional tie-breaking or warm-start heuristic for windowed/reduced search spaces.

Understanding that most do not transfer as hard constraints clarifies that the underlying problem structure was correctly captured; the additional QUBO terms compensate for the solver's approximate nature, not for gaps in the model itself.

4.2 Encoding of helper functions

Our helper functions are three: Goal approximation, Teleport penalty and backtracking penalty

4.2.1 Backtracking

The point of this penalty is that theoretically there is nothing wrong if you go back in order to find the correct answer. But if you backtrack that means that there is a more direct/ way to reach goal (without backtracking), so it would be nice for the solver to avoid moving backwards and find the path directly.

$$P_{backtracking} = K_{bt} \cdot \sum_{v \neq g_a} \sum_{t_1 < t_2} x_{v,t_1} x_{v,t_2} \tag{21}$$

Note that naturally this formula does not penalize repeating goal cell, else it would conflict with goal lock penalty/

4.2.2 Goal approximation

This is a modification of the goal penalty for the time-window approach. In this cases, the only window that will reach the goal is the last one; that is, most windows will not be able to satisfy the goal penalty. To solve this problem, the weight of the goal on the initial windows is reduced to avoid teleportation scenarios. Because the whole idea changes from reaching the goal to getting to a better position closer to the goal.

$$P_{goal_approx} = \sum_{t=1}^{T} \sum_{v \in V} [(P_{obs}(v) - \varphi(v,t)) \cdot x_{a,v,t}] \tag{22}$$

where P_{obs} is a soft obstacle avoidance coefficient using potential field. It benefits cells that have more connectivity and are from obstacles. Then there is φ, which is a piecewise function that was designed to be an **inverse-distance proximity function**

$$\varphi(v,t) = K_{goal_approx} \cdot \tau(t) \cdot \begin{cases} \dfrac{1}{0.7 + 0.165\, d(v)} & s = \text{enhanced_linear} \\ \dfrac{1}{1 + 1.2\, d(v)} & s = \text{exponential} \\ \dfrac{1}{1 + d(v)^{1.3}} & s = \text{quadratic} \\ \dfrac{1}{1 + \ln(1 + 2d(v))} & s = \text{logarithmic} \\ \dfrac{1}{c_1(G) + c_2(G)\, d(v)} & s = \text{adaptive} \\ \dfrac{1}{1 + 2\, d(v)} & s = \text{default} \end{cases} \tag{23}$$

where $\tau(t) = 1.2^{\frac{5t}{T}}$ and $d(v) = ||v - g_a||_1$

The idea is that it becomes large near the goal and small far away. With a negative sign this turn into a high reward for being near the goal vertex. Basically we refactor distance penalty to use smaller separations than normal Manhattan distances and keep the idea that goals should be considered as negative penalties (rewards).

Seen in another way, raw Manhattan distance $d(v)$ grows unboundedly with grid size, producing large, poorly-scaled QUBO coefficients. $\varphi(v,t)$ instead compresses distance into a bounded range via $1/(c+d)$-type transforms, giving the solver a tighter, more numerically stable energy landscape while preserving the same "closer is better" ordering.

4.2.3 Teleport Penalty

It is a helper function for adjacency penalty. It avoids the solver from considering profitable breaking adjacency rule in order to get to goal as sooner as possible

$$P_{tp} = K_{tp} \cdot \sum_{t=1}^{T-1} x_{a,g_a,t} \tag{24}$$

5 Software

Explain used software, links to dashboard and your implementation. Explain design choices

6 Benchmarking

Summarize your benchmarking.

7 Conclusion

Summarize what you done and what can be future work